IntelliSDLC

Software Requirements Specification

An AI-Assisted Tool for Managing Software Projects

Prepared by	Mihir Bhati
Document Version	2.0
Date	July 24, 2026

1. Introduction

1.1 What is this document?

This document explains what IntelliSDLC is, who it is for, and what it needs to do. It is meant for developers, reviewers, and anyone building or checking the project.

1.2 What is IntelliSDLC?

IntelliSDLC is a website that helps software teams manage their projects — from writing requirements, to designing, building, testing, and releasing software. On top of the normal project-tracking tools (like Jira), it adds AI features that help managers make better decisions, using real data from the project.

1.3 Who is this for?

Four types of users: Admins, Project Managers, Developers, and QA (testing) staff. Each one sees only the screens and actions they actually need.

1.4 Short forms used in this document

Short form	Meaning
SDLC	The stages a software project goes through: requirements, design, build, test, release
PM	Project Manager
QA	Quality Assurance (testing)
API	A way for two programs to talk to each other
ML	Machine Learning (a computer learning from data to make predictions)
JWT	A secure login token used to prove who a user is
CRUD	Create, Read, Update, Delete (the basic actions on any data)

2. Overview

2.1 The Project Lifecycle

IntelliSDLC follows a project from start to finish. The diagram below shows the stages, and where the AI features plug in.

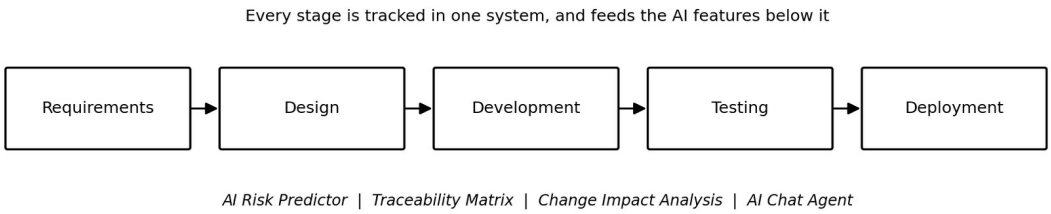


Figure 1: Project stages and where AI features apply

2.2 Who uses it, and how

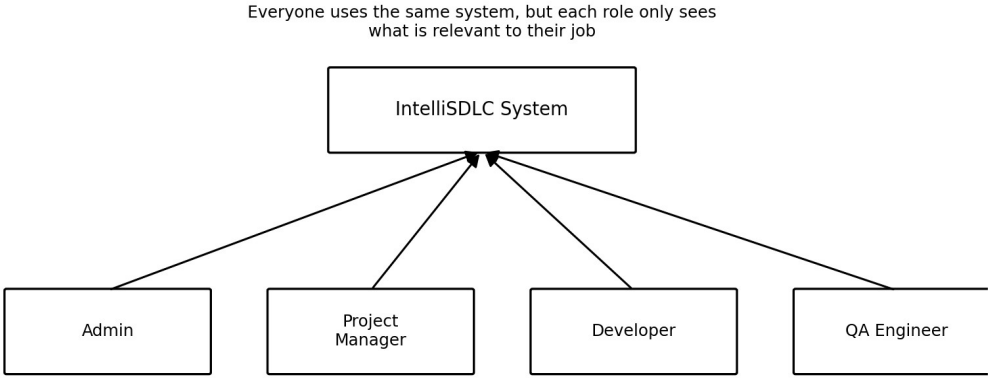


Figure 2: The four user types

Role	What they can do
Admin	Manage everything: users, projects, settings, reports
Project Manager	Create projects and sprints, assign work, view AI insights
Developer	View their tasks, update task status, log hours worked
QA Engineer	Manage test cases, log and track bugs, check test coverage

2.3 Where it runs

IntelliSDLC works in any standard web browser. The server side runs on Linux, using Docker so it can be started up the same way on any machine.

2.4 Assumptions and Limits

- The system needs internet access to reach the AI chat service (Claude or OpenAI).
- If real build data from GitHub isn't connected, sample/mock data is shown instead.
- The system is built for one organization at a time, not multiple companies sharing one setup.

3. What the System Does

The table below lists every feature. Must-have features are needed for the system to work at all. Good-to-have features make it stronger but can be added later if time runs short.

3.1 Core project-tracking features

Feature	What it does	Priority
Login and access control	Users log in securely; each role only sees what it needs	Must-have
Requirements	Write down what needs to be built; track changes over time	Must-have
Projects and sprints	Plan work into sprints; drag-and-drop task board; progress charts	Must-have
Tasks	Assign work to developers; log hours; see task history	Must-have
Bugs and testing	Log bugs, run test cases, see how much of the product is tested	Must-have
Releases	Track what version was released, when, and to which environment	Good-to-have

3.2 AI features (what makes this project different)

Feature	What it does	Priority
Risk Predictor	A trained AI model looks at bugs, delays, and workload to warn if a sprint will run late, and explains why	Must-have
Traceability Matrix	Automatically shows, for every requirement, whether it has been designed, built, tested, and released — no manual tracking needed	Must-have
Change Impact Analysis	When a requirement changes, shows everything else that might be affected (code, tests, bugs)	Good-to-have
Smart Task Assignment	Suggests the best developer for a new task, based on skills, experience, and workload	Good-to-have
AI Chat Assistant	A chat box where anyone can ask questions in plain English, like 'which requirements are untested?', and get an answer based on real project data	Must-have

How each AI feature is triggered:

Feature	User action leads to system response
Risk Predictor	User opens a sprint -> system calculates and shows a risk score with reasons
Traceability Matrix	User opens the matrix page -> system builds the table live from current data
Change Impact Analysis	User clicks 'Analyze Impact' on a requirement -> system lists everything affected
Smart Task Assignment	User picks a task type and skill while creating a task -> system suggests 3 developers
AI Chat Assistant	User types a question -> system gathers relevant data and replies in plain English

3.3 Supporting features

Feature	What it does
Design documents	Attach design files or links to a requirement, with version history
Change requests	A formal way to propose a change to a requirement; a manager approves or rejects it
Comments	Leave notes on any requirement, task, or bug
Attachments	Attach files, like bug screenshots, to any item
Notifications	Alerts when a task is assigned, a bug is filed, or a sprint is about to end
Activity log	A record of every important change, viewable by Admins
Reports	Downloadable summaries of project health and sprint progress
Search	Find any requirement, task, or bug by keyword
Dashboard	A home page showing the most important numbers, customized per role

4. How the System is Built

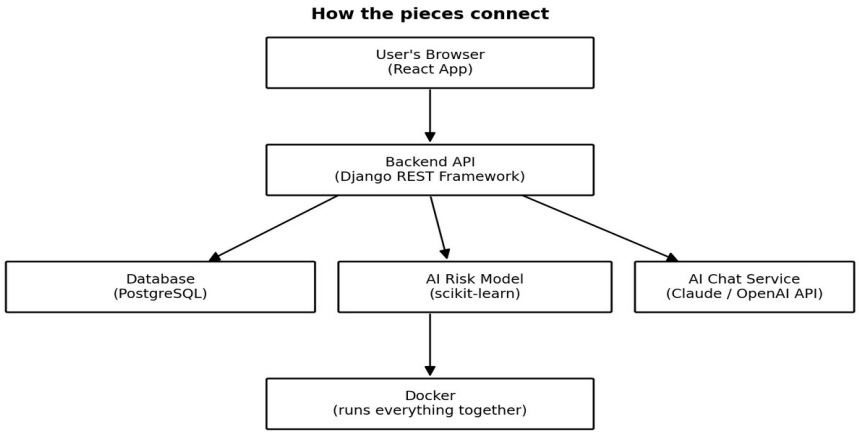


Figure 3: How the main parts connect

4.1 User Interface

A website built with React, that works on both computer and tablet screens.

4.2 Hardware Interface

No special hardware is needed — just a normal server (or cloud container) to run the app, and any device with a modern web browser to use it.

4.3 Software Interface

Part	Built with
What the user sees	React (a web front-end framework) with Tailwind for styling
Server logic	Django REST Framework (Python), handling logins and all data
Database	PostgreSQL, storing everything — requirements, tasks, bugs, etc.
Risk prediction	scikit-learn, a Python library for training simple AI models
Chat assistant	Claude or OpenAI's API, fed with real project data
Packaging	Docker, so the whole system starts the same way on any computer

4.4 Communication Interface

The system talks to itself over a secure web connection (HTTPS), and all data between the front end and the server is sent as JSON, a simple text format.

5. What Information is Stored

Below is a short list of the main things the database keeps track of. Each one is a table with rows of data.

Stored item	What it holds
Users	Login details and role for every person using the system
Requirements	What needs to be built, its status, and its version history
Projects / Sprints / Milestones	How work is grouped and scheduled
Tasks / Time logs	Individual pieces of work and hours logged against them
Developer profiles	Each developer's skills, experience, and workload
Bugs / Test cases	Problems found and the tests used to catch them
Deployments	What version was released, when, and where
Design documents	Files or links describing how something is designed
Change requests	Proposed changes to requirements, and their approval status
Comments / Attachments	Notes and files people add to items
Dependencies	Links between requirements, code, tests, and bugs (used for impact analysis)
Notifications / Activity log	Alerts sent to users, and a record of every important change

6. Other Requirements (Quality and Safety)

These are things the system must get right, beyond just having the right features.

Area	Requirement
Data safety	The database is backed up regularly so data is never permanently lost. Deleting something important always asks for confirmation first.
Security	Passwords are stored safely (never as plain text). Every action checks that the user is allowed to do it.
Speed	Normal actions (like saving a task) should feel instant — under 2 seconds.
Reliability	If an outside service (like the AI chat API) is down, the system shows a clear message instead of breaking.
Ease of use	Works well on both computer and tablet screens. Each role sees only what's relevant to them.
Easy to maintain	The system is built in clearly separated, versioned pieces so it's easy to update later.
Easy to set up anywhere	Because everything runs in Docker, the whole system can be started on a new computer in a few commands.
Documentation	All the technical endpoints are documented so any developer can understand how to use them.

7. Build Order (Summary)

A short version of the build order. A separate Action Plan document has the full week-by-week detail.

Step	What gets done
1	Set up the project, database, and Docker
2	Build login and access control
3	Build Requirements, Sprints, and the task board
4	Build Tasks, Bugs, and Deployments
5	Add design docs, change requests, comments, attachments
6	Build the main dashboard
7	Build and train the AI Risk Predictor
8	Build the Traceability Matrix
9	Build Change Impact Analysis (if time allows)
10	Build Smart Task Assignment
11	Build the AI Chat Assistant
12	Add notifications, activity log, reports, and search
13	Test everything and write the final documentation

Work Breakdown Structure

IntelliSDLC — AI-Augmented Software Development Life Cycle Platform

Project Planning and WBS – Gantt Chart – Scheduling

1. Introduction

IntelliSDLC is a project management platform purpose-built for software teams, covering the entire Software Development Life Cycle — requirements, design, sprints, tasks, bugs, testing, and deployment — inside a single system. Unlike a basic Jira/Trello clone, AI is wired directly into the core workflow through four decision-support features (a sprint risk predictor, an automated traceability matrix, a change-impact analyzer, and a smart developer-allocation engine), plus a conversational agent that answers questions from live project data rather than from general knowledge.

This document defines the Work Breakdown Structure (WBS) for the IntelliSDLC project: the deliverable-oriented hierarchical decomposition of the work required to build it, together with the WBS Dictionary describing each work package and the Gantt chart schedule derived from it.

2. What Is a Work Breakdown Structure?

A Work Breakdown Structure is one of the most important project management documents: it breaks a project's total scope of work into smaller, more manageable pieces, and in doing so ties together the project's scope, cost, and schedule baselines. The Project Management Institute's PMBOK Guide describes it as a deliverable-oriented hierarchical decomposition of the work to be carried out by the project team.

There are two common styles of WBS. A Deliverable-Based WBS organizes Level 1 around the project's major deliverables — the things the project produces — and decomposes each into the work packages needed to produce it. A Phase-Based WBS instead organizes Level 1 around the project's typical phases (e.g. Planning, Execution, Closure), with deliverables nested underneath each phase. In both styles, the lowest-level elements are Work Packages — units of work small enough to be owned and delivered by a single person or sub-team.

This document uses the Deliverable-Based approach, since IntelliSDLC's scope is most naturally organized around the distinct product modules it must deliver (the Foundation layer, the Collaboration layer, the AI layer, and so on) rather than around generic project phases.

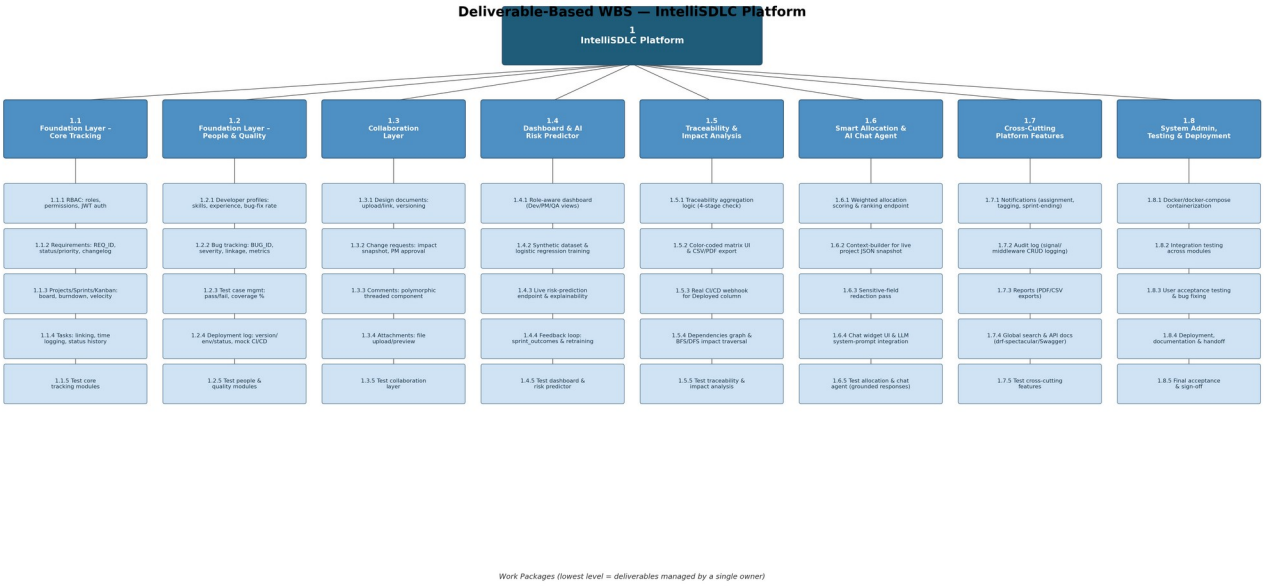
3. Methodology Used to Build This WBS

The WBS was developed using the standard iterative process:

- Gather critical project inputs — the IntelliSDLC feature specification, tech-stack decisions, and the list of gaps identified during planning (retraining loop, real CI/CD webhook, context redaction, missing Swagger task).
- Analyze these inputs to identify the deliverables the project must produce.
- Define the Level 1 elements — eight summary deliverables chosen so that, together, they capture 100% of the project's scope (the 100% Rule).
- Decompose each Level 1 element into unique, mutually exclusive Level 2 work packages, continuing until each package could reasonably be owned by a single developer or a small task.
- Create the WBS Dictionary — a narrative description of every work package, given in Section 5.
- Translate the Work Packages into a Gantt chart schedule for further scheduling and tracking, given in Section 6.

4. IntelliSDLC Work Breakdown Structure Diagram

The diagram below decomposes the IntelliSDLC Platform (Level 0) into eight Level 1 deliverables, each broken into five Level 2 work packages. Following the same convention used in the reference WBS, the final work package under every branch is the testing task for that deliverable.



5. WBS Dictionary

The WBS Dictionary below gives a narrative description of every work package shown in the diagram, organized by Level 1 deliverable.

1.1 Foundation Layer – Core Tracking

WBS ID	Deliverable / Work Package	Description
1.1.1	**RBAC, Roles & Authentication**	Four roles (Admin, PM, Developer, QA) with permissions enforced at the API level via custom permission classes and reflected in role-specific UI navigation; JWT-based login via djangorestframework-simplejwt.
1.1.2	**Requirements Management**	Requirement CRUD with auto-generated REQ_ID, status/priority/type fields, and a full version changelog capturing who changed what and when.
1.1.3	**Projects, Sprints & Kanban Board**	Project/Sprint data model, drag-and-drop Kanban board (To Do → In Progress → In Review → Done), auto-calculated burndown chart and sprint velocity rendered with Recharts.
1.1.4	**Tasks & Time Logging**	Tasks linked to a sprint and a requirement, with time logging and a full status-change history for audit purposes.
1.1.5	**Test Core Tracking Modules**	Unit and integration tests covering RBAC, requirements, sprints/Kanban, and tasks.

1.2 Foundation Layer – People & Quality

WBS ID	Deliverable / Work Package	Description
1.2.1	**Developer Profiles**	Skill tags, experience level, live active-task count, and a computed bug-fix rate that feeds the Smart Developer Allocation feature.
1.2.2	**Bug Tracking**	Auto-generated BUG_ID, severity levels, linkage to requirement/task/sprint, and rollup metrics (open count, critical count, average resolution time).
1.2.3	**Test Case Management**	Test cases linked to requirements with pass/fail/not-run status, driving test coverage percentages.
1.2.4	**Deployment Log**	Version, environment, and status tracking as a lightweight CI/CD log, with a mock JSON fallback when real CI/CD isn't wired up.
1.2.5	**Test People & Quality Modules**	Verification of developer-profile computations, bug metric rollups, and coverage calculations.

1.3 Collaboration Layer

WBS ID	Deliverable / Work Package	Description
1.3.1	**Design Documents**	Upload/link module tied to a requirement, versioned; the Traceability Matrix checks this table (instead of a free-text flag) to confirm a requirement is genuinely designed.
1.3.2	**Change Requests**	Submission triggers an automatic impact-analysis snapshot; a PM approves or rejects the request, and approval bumps the requirement's version — creating a real audit trail for scope changes.
1.3.3	**Comments**	A single polymorphic comments table (entity_type + entity_id) reused across requirements, tasks, and bugs through one threaded-comment UI component.
1.3.4	**Attachments**	File uploads on bugs, requirements, and tasks, with image previews and download links.

WBS ID	Deliverable / Work Package	Description
1.3.5	**Test Collaboration Layer**	Verification of design-document versioning, change-request workflow and version bumps, and the shared comment/attachment components.

1.4 Dashboard & AI Sprint Risk Predictor

WBS ID	Deliverable / Work Package	Description
1.4.1	**Role-Aware Dashboard**	Landing page tailored by role: Developers see their tasks, PMs see the full overview, QA sees bug/test metrics — plus health-summary cards, burndown chart, and recent-activity feed.
1.4.2	**Risk Model Training**	Synthetic 500+ row dataset with 9 correlated features; logistic regression trained with scikit-learn and saved via joblib.
1.4.3	**Live Risk-Prediction Endpoint**	Builds a real-time feature vector from the current sprint, scores it, and returns a risk score plus explainability (top contributing coefficients).
1.4.4	**Feedback Loop & Retraining**	A sprint_outcomes table records actual delay/no-delay outcomes after each sprint closes; a scheduled or admin-triggered job periodically retrains the model on this data, saving a new versioned model file and tracking accuracy over time.
1.4.5	**Test Dashboard & Risk Predictor**	Validation of role-based dashboard views and of model accuracy/explainability output.

1.5 Traceability Matrix & Change Impact Analysis

WBS ID	Deliverable / Work Package	Description
1.5.1	**Traceability Aggregation Logic**	Per-requirement checks: design document exists → Designed; linked task Done → Implemented; linked test case passing → Tested; covering deployment exists → Deployed.
1.5.2	**Matrix UI & Export**	Color-coded, auto-updating matrix with zero manual edits, exportable as CSV/PDF.
1.5.3	**Real CI/CD Webhook**	A minimal GitHub Actions webhook so the 'Deployed' column is authentically automated rather than relying solely on the mock JSON fallback; if skipped, documented as a known limitation.
1.5.4	**Change Impact Analysis (stretch goal)**	A dependencies table auto-populated on link creation; BFS/DFS traversal from a requirement node returns every affected module, test case, bug, and sprint. Optional — documented as future work if time is short.
1.5.5	**Test Traceability & Impact Analysis**	Verification that matrix cells update correctly and that graph traversal returns the correct affected-item set.

1.6 Smart Developer Allocation & AI Chat Agent

WBS ID	Deliverable / Work Package	Description
1.6.1	**Allocation Scoring Engine**	Deterministic weighted formula — score = 0.4×skill_match + 0.3×experience_score + 0.2×bug_fix_rate − 0.1×current_load_normalized — returning the top 3 ranked developers with score % and a human-readable reason.
1.6.2	**Chat Agent Context-Builder**	Assembles live JSON (sprint stats, bugs, workloads, requirement statuses) for a given project_id.

WBS ID	Deliverable / Work Package	Description
1.6.3	**Sensitive-Field Redaction Pass**	Strips or filters free-text fields such as comment bodies and attachment contents before the context reaches an external LLM API, with the exclusion explicitly documented.
1.6.4	**Chat Widget & LLM Integration**	The context is injected into a system prompt sent to the Claude/OpenAI API with an instruction to answer only from supplied context; floating chat widget supports message history and a typing indicator.
1.6.5	**Test Allocation & Chat Agent**	Confirms ranking output changes sensibly with workload, and verifies grounded responses by asking the same question before/after a data change.

1.7 Cross-Cutting Platform Features

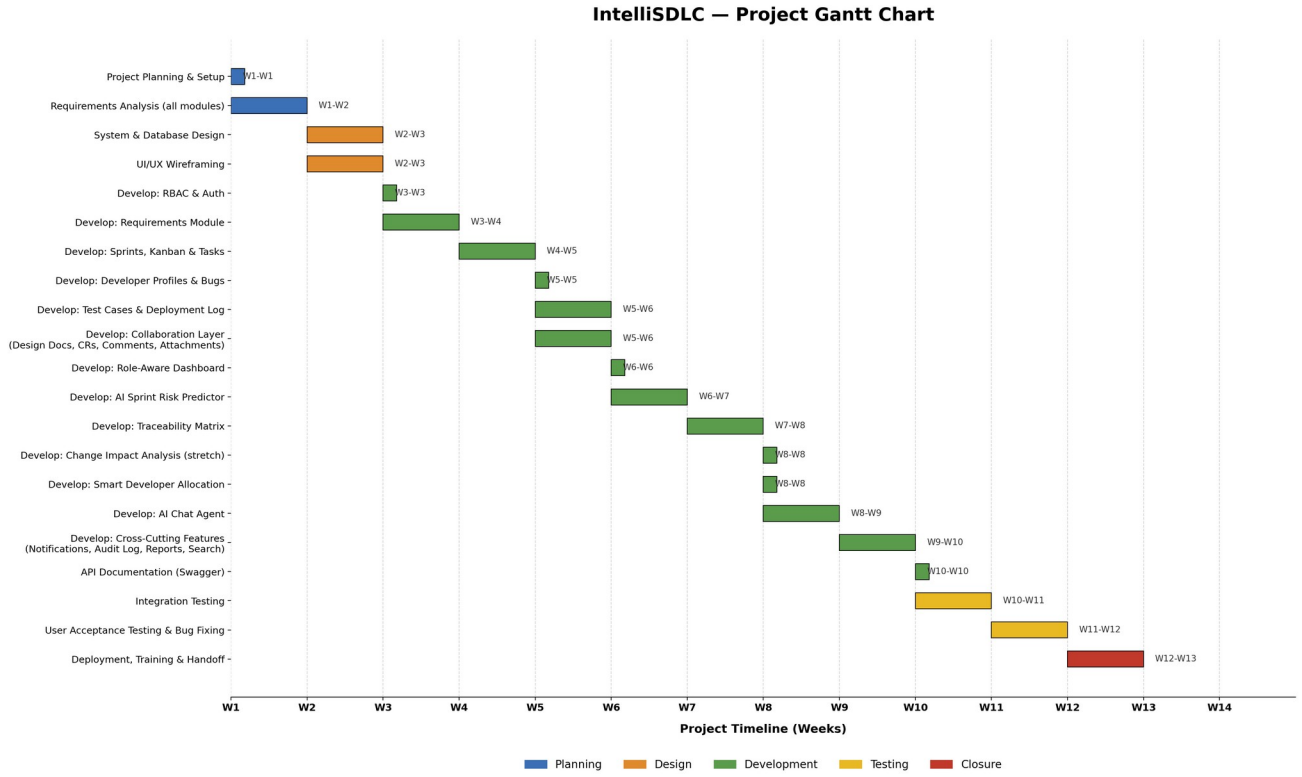
WBS ID	Deliverable / Work Package	Description
1.7.1	**Notifications**	Alerts for task assignment, being tagged in a bug, and a sprint ending within 2 days.
1.7.2	**Audit Log**	Signal/middleware-based logging of all create/update/delete actions on requirements, tasks, and bugs; viewable by Admins only.
1.7.3	**Reports**	Project Summary and Sprint Report exported as PDF; Traceability Matrix exported as CSV.
1.7.4	**Global Search & API Documentation**	Keyword search across requirements, tasks, and bugs; drf-spectacular wired in to expose /api/schema/ and a Swagger UI page, referenced in the README.
1.7.5	**Test Cross-Cutting Features**	Verification of notification triggers, audit-log completeness, report generation, and search accuracy.

1.8 System Admin, Testing & Deployment

WBS ID	Deliverable / Work Package	Description
1.8.1	**Containerization**	Docker / docker-compose setup so the whole stack — including the trained ML model file — comes up with a single docker-compose up command.
1.8.2	**Integration Testing**	End-to-end verification across the Foundation, Collaboration, Dashboard, and AI layers together.
1.8.3	**User Acceptance Testing & Bug Fixing**	Structured UAT pass against the original requirements, with fixes applied before closure.
1.8.4	**Deployment, Documentation & Handoff**	Final deployment, README/setup instructions, and API documentation handoff.
1.8.5	**Final Acceptance & Sign-Off**	Confirmation that all Level-1 deliverables meet the 100% Rule before project closure.

6. Project Schedule — Gantt Chart

The Work Packages above were sequenced into a 13-week schedule, respecting the dependency that the Collaboration layer, Dashboard, and AI features can only begin once the underlying Foundation-layer data model exists. Development phases are colour-coded by category (Planning, Design, Development, Testing, Closure), matching the reference Gantt format.



Schedule Detail

Work Package / Phase	Timeline	Category
Project Planning & Setup	W1	Planning
Requirements Analysis (all modules)	W1–W2	Planning
System & Database Design	W2–W3	Design
UI/UX Wireframing	W2–W3	Design
Develop: RBAC & Auth	W3	Development
Develop: Requirements Module	W3–W4	Development
Develop: Sprints, Kanban & Tasks	W4–W5	Development
Develop: Developer Profiles & Bugs	W5	Development
Develop: Test Cases & Deployment Log	W5–W6	Development
Develop: Collaboration Layer	W5–W6	Development
Develop: Role-Aware Dashboard	W6	Development
Develop: AI Sprint Risk Predictor	W6–W7	Development
Develop: Traceability Matrix	W7–W8	Development
Develop: Change Impact Analysis (stretch)	W8	Development
Develop: Smart Developer Allocation	W8	Development

Work Package / Phase	Timeline	Category
Develop: AI Chat Agent	W8–W9	Development
Develop: Cross-Cutting Features	W9–W10	Development
API Documentation (Swagger)	W10	Development
Integration Testing	W10–W11	Testing
User Acceptance Testing & Bug Fixing	W11–W12	Testing
Deployment, Training & Handoff	W12–W13	Closure

7. Conclusion

This Deliverable-Based WBS decomposes the IntelliSDLC platform into 8 Level 1 deliverables and 40 Level 2 work packages, satisfying the 100% Rule by explicitly covering every module described in the project brief — including the four scope gaps (model retraining loop, real CI/CD integration, chat-agent context redaction, and API documentation) that were folded into their respective branches rather than left as loose ends. Together with the WBS Dictionary and the derived Gantt schedule, this structure gives the project a single source of truth for scope, ownership, and timeline as development proceeds.